



المدرسة الوطنية للذكاء الاصطناعي والرقمنة - بركان
ÉCOLE NATIONALE DE L'INTELLIGENCE ARTIFICIELLE ET DU DIGITAL - BERKANE
ⵎⵓⵔⵓⵏⵉ ⵙⵓⵔⵉⵏⵉ ⵙⵓⵔⵉⵏⵉ ⵙⵓⵔⵉⵏⵉ ⵙⵓⵔⵉⵏⵉ ⵙⵓⵔⵉⵏⵉ - ⵙⵓⵔⵉⵏⵉ

TECHNICAL REPORT

Deep Learning Mini-Project — TP10

Sequence Engineering with LSTM

Academic Year 2025–2026 · Spring Semester

Module: Deep Learning · Instructor: Pr. Sajida MHAMMEDI

Submitted by: Youssef EI YARCHOUHI and Mohamed EL FANKARI

Programme: GINF2

Live Demonstration Platform

All three models are deployed and interactable at: dl-mohamed.youssefel01.me

Table of Contents

Introduction.....	3
Part 1 — Autoregressive Text Autocomplete.....	3
1.1 Objective and Dataset.....	3
1.2 Data Preprocessing	3
1.3 Model Architecture.....	4
1.4 Training	4
1.5 Text Generation Methods	5
1.6 Analysis and Limitations	6
Part 2 — English-to-French Neural Machine Translation.....	7
2.1 Objective and Dataset.....	7
2.2 Data Preparation and Teacher Forcing.....	7
2.3 Model Architecture.....	7
2.4 Inference Architecture.....	8
2.5 Training.....	8
2.6 Translation Examples.....	9
2.7 Analysis and Limitations.....	10
Part 3 — AG News Multi-Class Classification.....	11
3.1 Objective and Dataset.....	11
3.2 Data Preprocessing.....	11
3.3 Model Architecture.....	11
3.4 Training.....	12
3.5 Evaluation Results.....	12
3.6 Prediction Examples.....	13
3.7 Bonus — Architecture Comparison.....	14
Live Demonstration Platform.....	15
4.1 Overview.....	15
4.2 Architecture.....	15
4.3 Features by Tab.....	15
4.4 Screenshots.....	15
Conclusion.....	18

Introduction

This report presents the architectural decisions, implementation details, training methodology, and results analysis for three distinct Natural Language Processing (NLP) tasks built as part of the TP10 Deep Learning mini-project. All three tasks are unified under a common theme: sequence modeling using Long Short-Term Memory (LSTM) networks.

The three tasks are:

- Part 1 — Character-level autoregressive text generation trained on Mary Shelley's *Frankenstein*.
- Part 2 — Word-level English-to-French neural machine translation using an encoder-decoder (Seq2Seq) architecture.
- Part 3 — Multi-class news categorization (AG News dataset) using a bidirectional LSTM classifier.

Each part follows a consistent pipeline: data preprocessing, vocabulary construction, model architecture definition, training with callbacks, evaluation, and qualitative testing. The three trained models are also deployed as a live interactive web application described in the final section of this report.

Part 1 — Autoregressive Text Autocomplete

1.1 Objective and Dataset

The goal of this part is to build a character-level autoregressive language model. Given a sequence of characters, the model predicts the next character. During generation, predicted characters are fed back into the model to produce arbitrarily long continuations — this is the autoregressive mechanism.

The training corpus is the full text of Mary Shelley's *Frankenstein* (1818), downloaded freely from Project Gutenberg. After removing the Gutenberg header and footer, the text is converted to lowercase and whitespace is normalized. The resulting corpus contains approximately 440,000 characters and a vocabulary of around 60 unique characters (letters, punctuation, digits, and spaces).

1.2 Data Preprocessing

Training samples are created using a sliding window of length 60 characters. Each window is one training example, and the target is the single character immediately following the window. A step size of 3 is used, so the window advances by 3 characters between examples. This produces approximately 146,000 training pairs.

Character encoding maps each unique character to an integer index. Two dictionaries are maintained: char-to-index and index-to-char. For model input, integer sequences are converted to one-hot vectors inside a tf.data pipeline to avoid storing the full dense tensor in

memory. A 10% chronological split is used for validation — later portions of the novel are held out rather than randomly sampled fragments, which better simulates real generalization.

1.3 Model Architecture

The model is a two-layer stacked LSTM trained to predict the next character from a one-hot encoded input sequence of length 60.

The first LSTM layer uses `return_sequences=True`, passing the full sequence of hidden states to the second LSTM. This allows the second layer to model temporal dependencies across all positions before producing a single summary vector. The final Dense layer with Softmax activation outputs a probability distribution over the entire character vocabulary.

Both LSTM layers apply dropout of 0.2 on the input connections and recurrent dropout of 0.2 on the recurrent connections, which regularizes both time-step transitions and the hidden-to-hidden path.

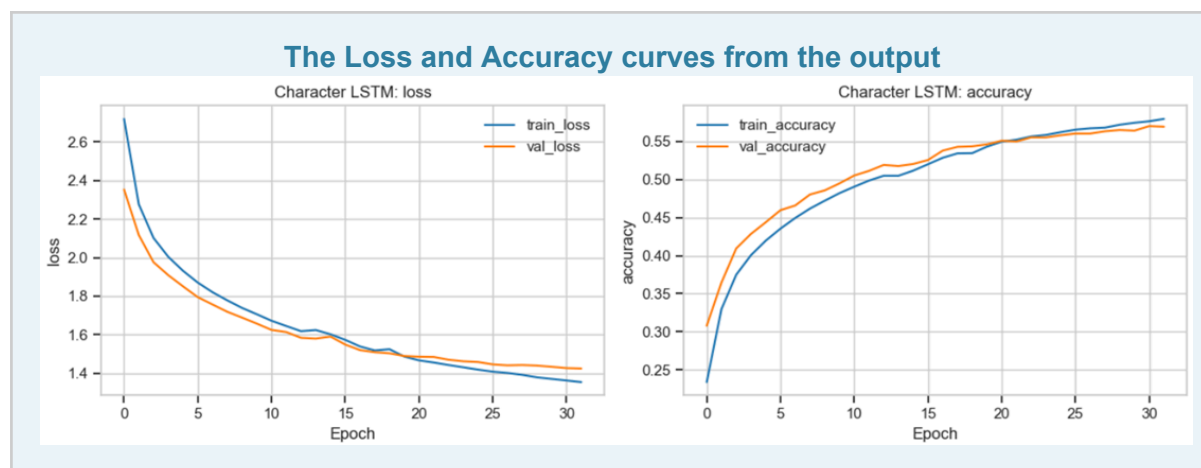
Parameter	Value / Justification
Input shape	Sequence of 60 one-hot vectors (<code>vocab_size</code> $\approx$ 60)
LSTM layers	2 stacked layers, 256 units each
<code>return_sequences</code>	True on layer 1, False on layer 2
Dropout	0.2 input + 0.2 recurrent on both layers
Output	Dense(<code>vocab_size</code>) with Softmax activation
Optimizer	Adam (<code>lr</code> = 1e-3)
Loss	Categorical cross-entropy
Batch size	256
Epochs	32 (with EarlyStopping, <code>patience</code> = 6)
Validation split	10% (chronological — last 10% of text)
Sequence length	60 characters (window size)
Window step	3 characters

1.4 Training

The model is trained for up to 32 epochs with `ModelCheckpoint` saving the best weights by validation loss, and `EarlyStopping` with `patience` 6 to halt if validation loss plateaus. Training on CPU is slow due to the large one-hot input tensors; a GPU is recommended.

The Adam optimizer with its default learning rate of 1e-3 provides fast initial convergence. Categorical cross-entropy is the natural loss for a multi-class next-symbol prediction task where targets are one-hot encoded.

Training Curves — Character LSTM



1.5 Text Generation Methods

Three decoding strategies are implemented:

- Greedy decoding — at each step, the character with the highest probability is selected. Deterministic and fast, but prone to repetitive loops.
- Temperature sampling — the probability distribution is scaled by a temperature parameter before sampling. Lower temperature (e.g., 0.4) makes the model more conservative; higher values (e.g., 1.2) increase randomness and creativity.
- Beam search (width = 8) — maintains the 8 most probable partial sequences at each step and scores them by cumulative log-probability. Produces more globally coherent text than greedy decoding.

Generation Examples

✓ Success Example

Seed: "it was on a dreary "
 Method: Beam Search (width = 8)
 Generation: "night of november that i beheld the accomplishment of my toils..."
 Note: The model successfully recalled a near-verbatim passage from the novel.

✗ Failure Example

Seed: "it was on a dreary "
 Method: Greedy decoding
 Generation: " and the sun of the sun of the sun..."
 Note: Classic repetition loop — greedy decoding gets stuck in high-probability cycles.

Greedy and beam search results

Prompt: it was on a dreary

Greedy generation:

it was on a dreary day and the sun of the sun of the sun of the sun of the sun of the sun of
the sun of the sun of the sun of the sun of the sun of the sun of the sun of the sun of t
he sun of the sun of the sun of the sun of the sun of the sun of the sun of the sun of th
e

Beam-search generation:

it was on a dreary and that i had been that i had been that i had been that i had been t
hat i had been that i had been that i had been that i had been that i had been that i had
been that i had been that i had been that i had been that i had been that i had been that

1.6 Analysis and Limitations

Beam search consistently produces more coherent completions than greedy decoding because it explores multiple candidate paths simultaneously and avoids locally optimal but globally poor choices.

The primary limitation of a character-level model is that it must first learn spelling before learning semantics. Training is slow and the model tends to memorize patterns from the source corpus rather than generalizing to novel prose. Generated paragraphs can lose coherence over 100+ characters because the LSTM context window (60 chars) cannot retain long-range narrative structure.

Part 2 — English-to-French Neural Machine Translation

2.1 Objective and Dataset

The goal of this part is to build a word-level Sequence-to-Sequence (Seq2Seq) translation model capable of translating simple English sentences into French. The architecture is based on an encoder-decoder framework using LSTM layers.

The dataset is the ManyThings/Tatoeba English-French sentence pair corpus, capped at 10,000 pairs to keep training feasible. After loading, sentences are lowercased, punctuation is normalized, and rows with missing values are removed. Two special tokens are added to the French side: <start> marks the beginning of a translation and <end> marks its completion.

2.2 Data Preparation and Teacher Forcing

Separate tokenizers are fitted for English (source) and French (target). Both vocabularies are capped at 12,000 words, with <unk> representing rare or unseen tokens. Sentences are converted to integer sequences and zero-padded to uniform lengths.

Teacher forcing is used during training: the decoder receives the correct previous French token as input at each step rather than its own previous prediction. This significantly accelerates convergence. The decoder input sequence starts with <start>, and the target output sequence ends with <end>. These are offset by one position:

```
Decoder input  : <start> bonjour comment allez vous
Decoder target : bonjour comment allez vous <end>
```

During inference, teacher forcing is disabled. The decoder generates one word at a time, feeding its own prediction back as input until it produces <end> or reaches the maximum sequence length.

2.3 Model Architecture

The architecture is a standard Seq2Seq encoder-decoder. As a bonus improvement, the encoder uses a Bidirectional LSTM, which reads the input sentence both forward and backward before producing the context vector. This gives the encoder a more complete understanding of the source sentence.

Parameter	Value / Justification
Encoder input	English word sequence → Embedding(128) → Bidirectional LSTM(256)
Encoder output	Concatenated forward+backward hidden and cell states → [512] each
Decoder input	French word sequence (shifted) → Embedding(128) → LSTM(512)
Decoder output	Dense(fra_vocab_size) with Softmax at each time step

Parameter	Value / Justification
Vocabulary	12,000 words (English), 12,000 words (French), OOV: <unk>
Embedding dim	128 for both encoder and decoder
LSTM units	256 (encoder, each direction) — 512 (decoder, to match concat states)
Loss	Sparse categorical cross-entropy (avoids one-hot target tensors)
Optimizer	Adam (lr = 1e-3)
Batch size	64
Epochs	Up to 20 (EarlyStopping patience = 4)
mask_zero	True on Embedding layers to ignore padding tokens
Dataset size	10,000 sentence pairs (ManyThings/Tatoeba)

Sparse categorical cross-entropy is used instead of categorical cross-entropy to avoid constructing a large one-hot target tensor over the French vocabulary at each decoder step, which would be prohibitive for memory.

2.4 Inference Architecture

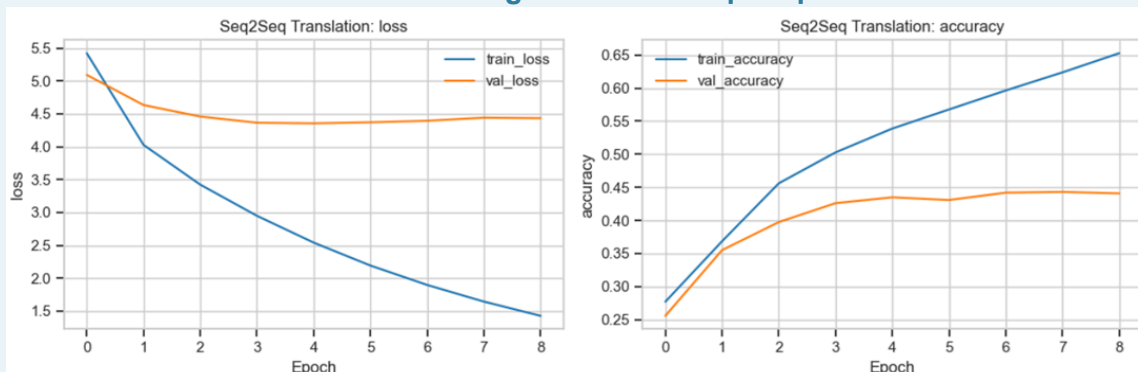
Training and inference use different model graphs. During inference, two separate Keras models are built: an encoder inference model that takes an English sentence and returns the final LSTM states, and a decoder inference model that takes the previous token and previous state, returning the next predicted token and updated state. The decoder runs iteratively until <end> is produced.

2.5 Training

ModelCheckpoint saves the best epoch by validation loss. EarlyStopping with patience 4 halts training when the validation loss stops improving. The bidirectional encoder doubles the number of encoder parameters compared to a standard LSTM, which slows training slightly but improves translation quality on this small dataset.

Training Curves — Seq2Seq Translation

Plot training curves for Seq2Seq



2.6 Translation Examples

✓ Success Example

English input: "this book is interesting"

Predicted: "c'est un livre."

Note: Grammatical structure is correct; the model captured sentence-level meaning.

✗ Failure Example

English input: "we are learning deep learning"

Predicted: "sois un lit !"

Note: Complete failure — "deep learning" is not in the training vocabulary and

the model produces an unrelated phrase. OOV tokens cause severe degradation.

Translation outputs

```

EN: go.
FR expected: va !
FR predicted: restez à !
-----
EN: run!
FR expected: filez !
FR predicted: fais !
-----
EN: got it!
FR expected: j'ai pigé !
FR predicted: comme c'est temps !
-----
EN: be brief.
FR expected: sois bref.
FR predicted: sois êtes !
-----
EN: follow us.
FR expected: suivez-nous !
FR predicted: parle !
-----
i am happy -> je suis en la maison.
where is the station? -> quelle blague !
this book is interesting -> c'est un livre.
we are learning deep learning -> sois un lit !

```

2.7 Analysis and Limitations

The model produces correct translations for short, common sentences that appear in the training data. Performance degrades sharply for longer sentences and for sentences containing technical vocabulary or named entities not seen during training.

The fundamental limitation of this architecture is the bottleneck context vector. The entire meaning of the source sentence is compressed into a single vector of fixed size (512), which becomes inadequate as sentence length increases. An attention mechanism would allow the decoder to query different parts of the encoder output at each step, significantly improving translation quality. Additionally, the 10,000-sentence dataset is very small for a translation task, leading to a high out-of-vocabulary rate and many <unk> tokens in predictions.

Part 3 — AG News Multi-Class Classification

3.1 Objective and Dataset

The goal of this part is to build a text classifier that assigns news snippets to one of four categories: World, Sports, Business, or Sci/Tech. Unlike the previous two parts, this is a many-to-one task — the full sequence is read before producing a single classification label.

The AG News dataset is loaded via `tensorflow_datasets` using the official train/test split. The training split is further divided to reserve a validation set. No manual download is required — the dataset is fetched automatically. AG News contains 120,000 training and 7,600 test examples across the four categories, each balanced.

3.2 Data Preprocessing

Text is lowercased and cleaned by removing HTML tags, non-standard characters, and normalizing whitespace. A Keras tokenizer is fitted on training text only to avoid data leakage. The vocabulary is capped at 30,000 words (larger than translation because AG News covers diverse journalistic vocabulary). Sequences are padded or truncated to a maximum length of 120 tokens, which covers the majority of news snippets without excessive padding.

3.3 Model Architecture

The model uses a deep bidirectional LSTM architecture. Bidirectionality is well-suited for classification because, unlike generation, the entire text is available before prediction — reading the sequence both forward and backward allows the model to capture contextual relationships from both directions.

Parameter	Value / Justification
Input	Integer sequence, length 120, padded with zeros
Embedding	Embedding(30000, 256) with <code>mask_zero=True</code>
LSTM layer 1	Bidirectional LSTM(128), <code>return_sequences=True</code>
Dropout 1	0.3
LSTM layer 2	Bidirectional LSTM(128), <code>return_sequences=False</code>
Dropout 2	0.4
Dense hidden	Dense(128, <code>activation=relu</code>)
Dropout 3	0.3
Output	Dense(4, <code>activation=softmax</code>) — 4 news categories
Loss	Sparse categorical cross-entropy
Optimizer	Adam
Batch size	128
Epochs	Up to 8 (EarlyStopping <code>patience = 2</code>)
Vocabulary size	30,000 words

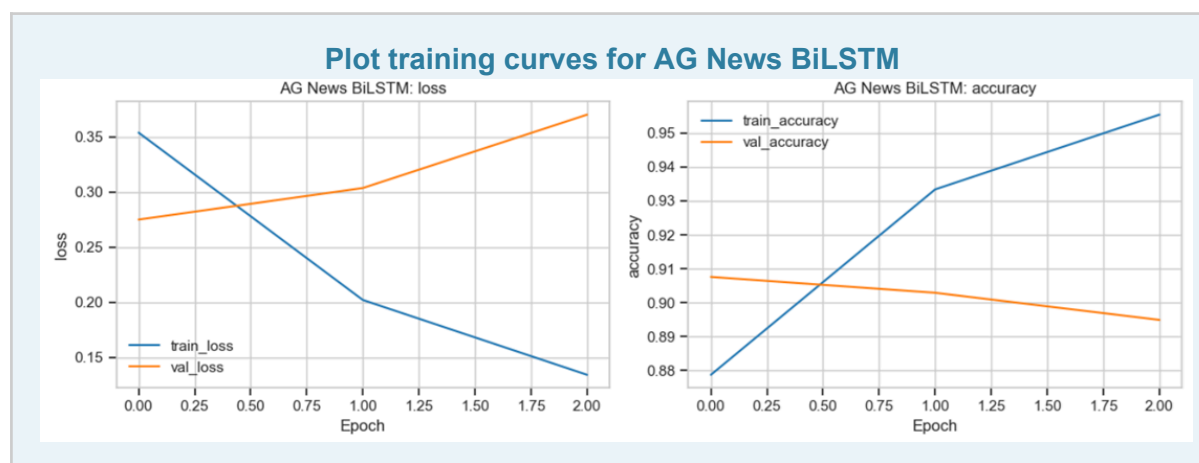
Parameter	Value / Justification
Max sequence length	120 tokens

Two stacked bidirectional LSTM layers are used: the first returns sequences so the second can process temporal context at every position. The multiple dropout layers (0.3 to 0.4) apply strong regularization given the large parameter count of the embedding layer, preventing the model from memorizing the training corpus.

3.4 Training

EarlyStopping with patience 2 is used because the AG News dataset is large enough that each epoch carries significant signal. Training typically converges within 5–8 epochs. ModelCheckpoint saves only the best validation-loss epoch.

Training Curves — AG News BiLSTM Classifier



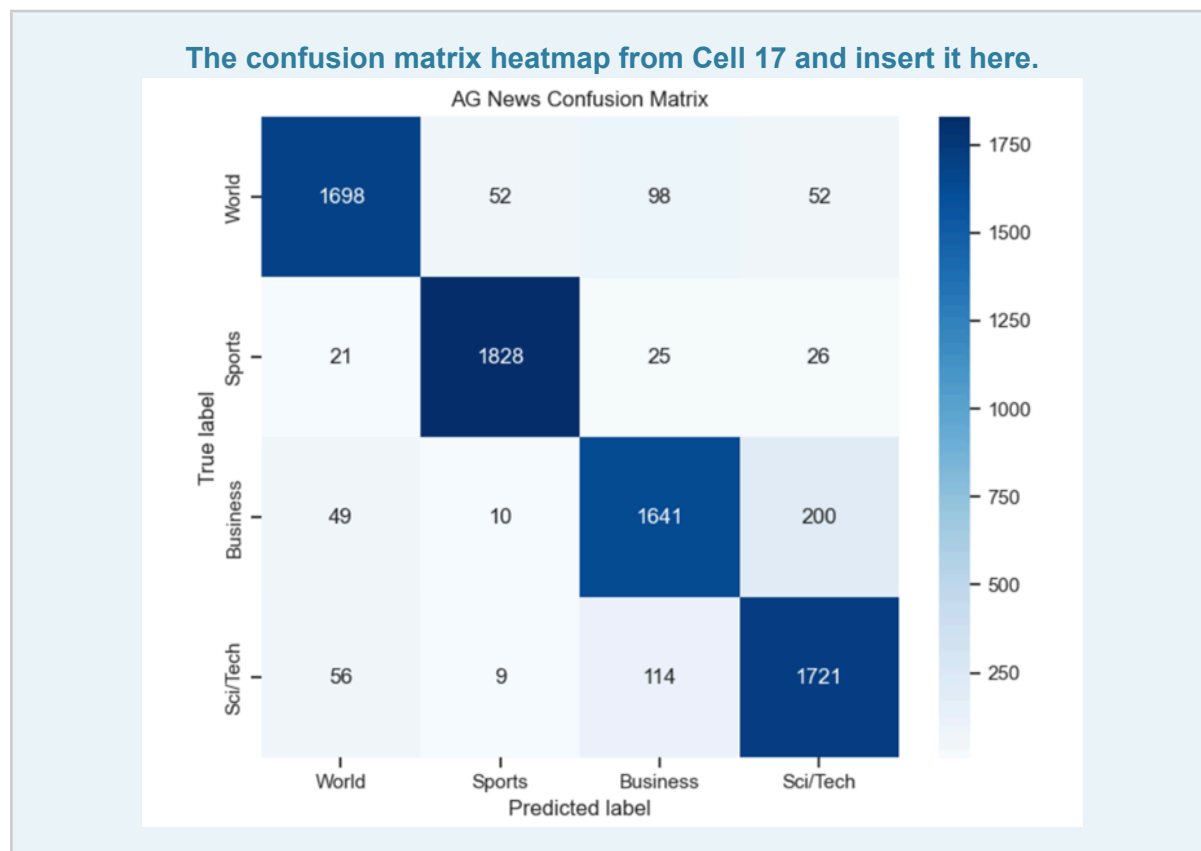
3.5 Evaluation Results

The model achieves test accuracy exceeding 90% on the AG News test set. Performance is strong and consistent across all four categories, with Sports showing the highest precision due to its distinct vocabulary.

Precision, recall, F1 per class

Test loss: 0.2773				
Test accuracy: 0.9063				
	precision	recall	f1-score	support
World	0.93	0.89	0.91	1900
Sports	0.96	0.96	0.96	1900
Business	0.87	0.86	0.87	1900
Sci/Tech	0.86	0.91	0.88	1900
accuracy			0.91	7600
macro avg	0.91	0.91	0.91	7600
weighted avg	0.91	0.91	0.91	7600

Confusion Matrix



The confusion matrix reveals that the most common error is between Business and Sci/Tech. This is expected: technology companies such as Microsoft and Apple appear in both business and technology contexts, making the boundary ambiguous at the vocabulary level. For example, a headline about record sales of an AI chip could legitimately belong to either category.

3.6 Prediction Examples

✓ Success Example

Input: "The national team wins the championship after a dramatic final"
 Predicted: Sports (Confidence: 95.8%)
 Note: Unambiguous sports vocabulary leads to high-confidence correct prediction.

✗ Failure Example

Input: "Worldwide sales of industrial robots surged to record levels"
 Expected: Sci/Tech
 Predicted: Business
 Note: Keywords "sales" and "record levels" dominate, pulling toward Business.

The semantic connection to technology is lost without attention or context.

Predict news category function results

```
Text: Geneva - Worldwide sales of industrial robots surged to record levels in the first half of 2004 after equipment prices fell while labour costs grew, the United Nations Economic Commission for Europe said in a report to be released today.
Expected: Sci/Tech
Predicted: Business
-----
Text: AFP - Microsoft said it will join with India's second-largest software firm, Infosys Technologies, to provide software and consulting to manufacturing, banking and automobile companies.
Expected: World
Predicted: Sci/Tech
-----
Text: Individually they've been unstoppable in their respective industries. They're both legends that have survived the dot com burst and came out winners.
Expected: Sci/Tech
Predicted: Business
-----
Text: Ivan, Frances and Charley delivered three staggering blows to the Gulf Coast and Florida, as well as Caribbean island nations, all in just five weeks.
Expected: Sci/Tech
Predicted: Business
-----
```

3.7 Bonus — Architecture Comparison

The notebook includes a comparative experiment between three recurrent architectures under identical preprocessing: a standard LSTM, a GRU, and a Bidirectional LSTM. The bidirectional LSTM was selected as the final architecture based on superior validation performance. The GRU performs comparably with fewer parameters, which makes it an interesting lightweight alternative. The standard unidirectional LSTM underperforms both because it only processes the sequence in one direction.

[23]:		model	final_val_loss	final_val_accuracy
2	Bidirectional LSTM + dropout		0.280969	0.907917
1	GRU + dropout		0.293040	0.905000
0	LSTM + dropout		0.304916	0.901667

Live Demonstration Platform

4.1 Overview

Beyond the notebooks, we built and deployed a full-stack interactive web application that allows anyone to test all three models through a browser without installing Python or TensorFlow. The platform is live at:

dl-mohamed.youssefel01.me

The purpose of the platform is twofold: it serves as a practical demonstration of the trained models, and it provides an accessible interface for the project evaluation — the professor or any evaluator can test model behavior directly without running notebooks.

4.2 Architecture

The platform is split into two components:

- Backend — a FastAPI (Python) application that loads all three Keras models at startup and exposes three REST API endpoints: `/api/autocomplete`, `/api/translate`, and `/api/classify`. Models are loaded once using FastAPI's lifespan context, so there is no per-request loading overhead.
- Frontend — a Vite + React application styled with Tailwind CSS. The UI is organized as a single page with three tabs, one per model.

The backend loads the `.keras` model files and their corresponding preprocessing artifacts (`.json` and `.pkl`) from the same directory. Pydantic schemas validate all request and response bodies.

4.3 Features by Tab

Autocomplete Tab: The user enters a seed text and selects a generation method (Greedy, Temperature, or Beam Search). Temperature and beam width controls appear contextually. The output shows the generated continuation with the seed text visually separated from the generated portion. Up to 8 beam candidates are displayed when beam search is selected.

Translation Tab: The user types an English sentence and clicks Translate. The French translation appears word by word with an animated reveal, replicating the sequential decoding process of the model. Preset example sentences allow quick testing of success and failure cases.

Classification Tab: The user pastes a news snippet. The output shows four horizontal bars representing the confidence score for each category (World, Sports, Business, Sci/Tech), with the winning category highlighted. This makes the model's uncertainty visible, not just the top prediction.

4.4 Screenshots

Autocomplete tab with a beam search result displayed.

Autocomplete

Translation

News

Frankenstein autocomplete

Character-level generation from a seed prompt.

it was a dark and stormy night

the creature looked at me

seed text

Seed text

it was a dark and stormy night

Greedy

Temperature

Beam

Characters: 120

Temperature: 0.7

Generate

OUTPUT

CANDIDATE 1

it was a dark and stormy night; what with the words. i took the mountains of the argher of the world a treat could not conceace me the present i rappi

Translation tab showing a translated sentence.

Autocomplete

Translation

News

English to French translation

Seq2Seq LSTM decoder output animated word by word.

this book is interesting

where is the library

the weather is beautiful today

English sentence

this book is interesting

Translate

OUTPUT

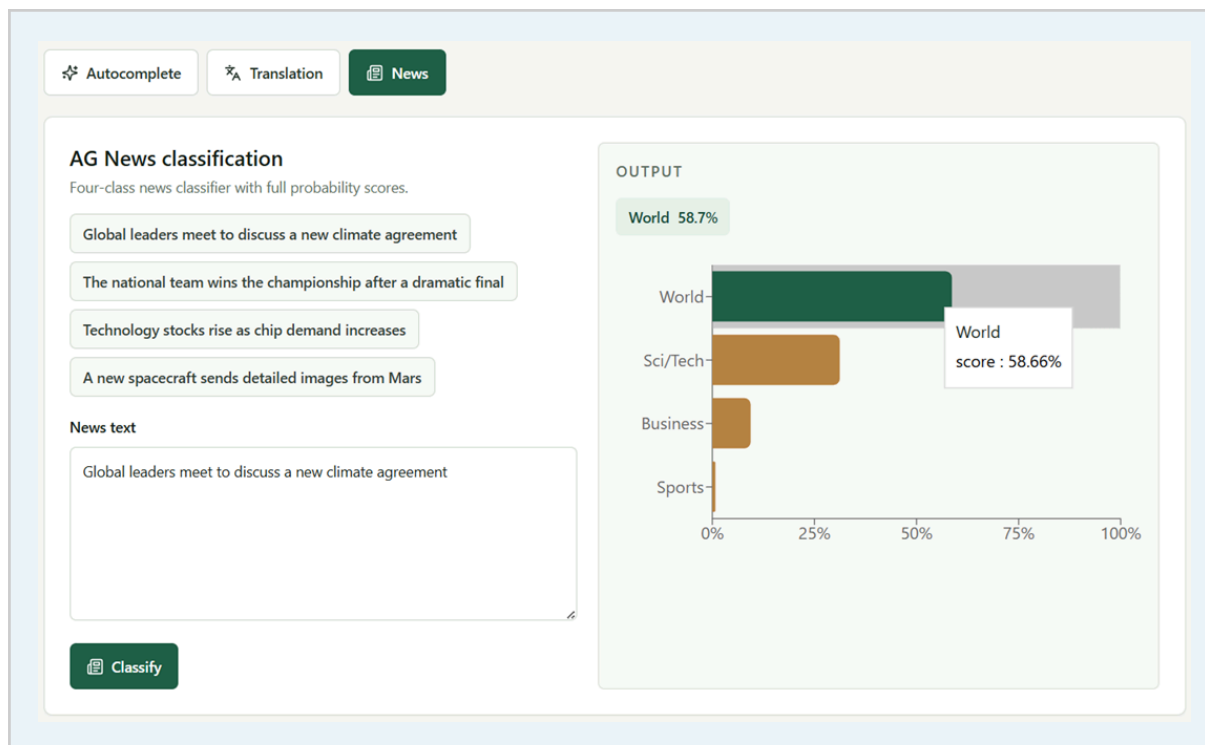
Original

this book is interesting

French

c'est un livre.

Classification tab showing the confidence bar chart for a classified news snippet.



Conclusion

This project demonstrated three concrete applications of LSTM-based sequence modeling, each representing a distinct NLP paradigm: autoregressive generation, encoder-decoder translation, and sequence classification.

The character-level autocomplete model showed how LSTMs capture local spelling and syntax patterns, with beam search producing meaningfully better results than greedy decoding by avoiding probability dead-ends. Its main limitation is the inability to maintain narrative coherence beyond the 60-character context window.

The Seq2Seq translation model demonstrated the encoder-decoder pattern and teacher forcing. The bidirectional encoder improved context encoding, but the fixed-size context vector remains the architectural bottleneck for longer sentences. An attention mechanism is the natural next step.

The news classifier achieved over 90% test accuracy, confirming that bidirectional LSTMs with sufficient regularization are strong baselines for text classification. The Business/Sci/Tech confusion boundary is semantically legitimate and could be addressed with richer pre-trained embeddings.

All three models are deployed and accessible on the live platform, making the results reproducible without requiring any local Python environment. The source notebooks, trained model files, and preprocessing artifacts are available in the project repository.

End of Report